

SIH 1454

PHISHSHIELD AI

Presentation + Jury Viva Master Guide

Intelligent detection of phishing domains imitating genuine domains

Based on the current CyberGuard repository. Do not invent metrics.

1. How to use this book

The SIH name is **PhishShield AI**. The code is branded **CyberGuard**. Say both in viva.

Accuracy rule: The running stack is FastAPI + SQLite + a rule engine + live threat lists. There is no Flask, no PostgreSQL server, no TensorFlow/CNN, and no published F1/accuracy.

Full markdown companions:

- docs/IMPLEMENTATION_REALITY_CHECK.md
- docs/TECH_STACK_STUDY_GUIDE.md
- docs/PRESENTATION_MASTER_GUIDE.md
- docs/JURY_VIVA_QUESTION_BANK.md (120 questions)
- docs/TEAM_STUDY_PLANS.md

Must know: Website and extension both call POST `/api/v1/threat/check-url` with `{ url, view: "both" }`. Same URL, same score.

2. Implementation reality check

Feature	Status	Evidence
FastAPI + Uvicorn	IMPLEMENTED	backend/app/main.py
Flask	NOT IMPLEMENTED	not in requirements
React + Vite + Tailwind + Axios + Recharts	IMPLEMENTED	frontend/package.json
SQLite threats.db	IMPLEMENTED	database/connection.py
PostgreSQL	NOT IMPLEMENTED	planned scale-up
SQLAlchemy at runtime	PARTIAL	models exist, routes use sqlite3
Rule URL engine	IMPLEMENTED	url_checker.py
Psychology urgency/fear/greed	IMPLEMENTED	psychology.py
Simple + Technical views	IMPLEMENTED	warnings.py
Google Safe Browsing / OpenPhish	IMPLEMENTED	external_intel.py
Aggressive ads + SSRF guards	IMPLEMENTED	ad_signals.py
CNN / sklearn / TensorFlow	NOT IMPLEMENTED	no model files
ml_confidence	PARTIAL	column default 0.0
JWT login	IMPLEMENTED	user.py, auth.py
Auth required on scan	NOT IMPLEMENTED	check-url is public
Extension MV3	IMPLEMENTED	extension/
Message scanner UI	PARTIAL	API yes, React unused
Rate limiting	NOT IMPLEMENTED	admit this gap
Published accuracy / F1	NOT AVAILABLE	never invent a number

Do not say: We use Flask, PostgreSQL, a CNN, 99% accuracy, federated learning, or zero-knowledge proofs.

3. Architecture

```
Extension / React
  REST JSON   POST /api/v1/threat/check-url
  FastAPI :8000
    URLChecker + psychology + intel + ads
    SQLite threats.db
    JSON: score, simple_view, technical_view
```

Beginner: You paste a link or open a website. Our server checks it and warns you in simple English.

Technical: Multi-signal rule engine plus live lists (Safe Browsing if a key is set, else OpenPhish), DNS, RDAP age, brand impersonation, Hinglish psychological phrases, and aggressive-ad HTML sampling. React and the extension are clients of one FastAPI process.

Jury tip: If they draw Flask to Postgres to CNN, redraw FastAPI to SQLite to rules. Offer Postgres and a trained model as the roadmap.

4. check-url pipeline

1. Pydantic body: url, view = both / simple / technical
2. Trim; add https:// if missing
3. Analyze: DNS, intel, age, phishing fragments, piracy, ads, brand, TLD, keywords, IP, length, HTTPS, psychology
4. Cap score at 100. LOW under 30, MEDIUM under 50, HIGH under 70, else CRITICAL
5. Trusted sites such as google.com return 0 if the domain exists
6. Insert url_checks (user_id not set; ml_confidence stays 0)
7. HIGH/CRITICAL also insert threats
8. Return Simple View and/or Technical View

Other APIs: /health, /api/v1/threat/check-message (no dashboard UI), stats, signup, login, profile.

Warning: average_response_time_ms = 245 is hardcoded. Do not call it a measured SLA.

5. Risk points actually in code

Check	Points
Domain does not exist	+35
Google Safe Browsing / OpenPhish flagged	+40
Domain age under 30 / 90 days	+20 / +10
Known phishing fragment (paypa1, amaz0n)	+40
Piracy / scam streaming name	+35
Brand impersonation	+40
Suspicious TLD (.xyz, .tk)	+15
Suspicious keyword (login, verify)	+5
IP-based host	+30
Too many subdomains	+10
Very long URL	+8
No HTTPS	+5
Email used as host	+20
Urgency / fear / greed language	+15 / +15 / +12
Aggressive popup / malware ads	+20

6. Database (SQLite, not PostgreSQL)

Table	Purpose
users	username, email, bcrypt password hash
url_checks	scanned URL, score, level; user_id unused
threats	HIGH/CRITICAL rows for charts

SQL injection defense: parameterized ? queries. Two users at once: SQLite serializes writes. Database down: API 500, extension says unavailable.

7. Machine learning honesty

METRIC NOT YET AVAILABLE. No dataset file, no train/test split, no confusion matrix, no CNN.

If asked for ML theory: accuracy alone is dangerous because a dummy “always SAFE” model can look accurate. Recall matters more for phishing. A future model can use the same features we already score by rules. The `ml_confidence` column is reserved.

Must know: “Act now” on a real sale is not automatically phishing. We add points. CRITICAL needs several strong signals.

8. Security of our system

Issue	Status
SQL injection	IMPLEMENTED — parameterized SQL
Password hashing	IMPLEMENTED — bcrypt
JWT	IMPLEMENTED — change SECRET_KEY in production
SSRF on ad fetch	IMPLEMENTED — block private IPs, timeout, size cap
XSS	PARTIAL — React escape + extension escapeHtml
CORS	WEAK — allow all origins
Rate limiting	NOT IMPLEMENTED
Full TLS certificate check	NOT IMPLEMENTED — only https:// prefix
Reading page passwords	Not done, by design

9. Extension

Manifest V3. Auto-scans the tab. Same API as the website. Badge: OK / MED / HIGH / CRIT. On-page risk card. Full warning on HIGH/CRITICAL. If the backend is down it says **UNAVAILABLE**, never SAFE.

10. 2-minute pitch

Phishing copies banks and UPI apps. HTTPS and logos are not proof. PhishShield AI, in our CyberGuard repo, scores any URL with the same FastAPI engine from the website and the Chrome extension. We combine live intel, brand impersonation, Hinglish urgency language, and shady popup ads into a 0–100 score and a plain-English warning. This version is a multi-signal rule engine plus threat lists, not a trained CNN, and it stores scans in SQLite. Next: a supervised model and PostgreSQL. Today the value is consistent, explainable, real-time protection without invented results.

11. 20 slides

Title → Problem → Gap → Solution → Innovation → Architecture → Pipeline → Intelligent detection (current + planned) → Psychology → Simple View → Technical View → Extension → Data model → Security → Testing → Stack → Novelty → Limitations → Future → Demo.

12. Cheat sheet

- Not Flask — FastAPI
- Not Postgres yet — SQLite
- Not CNN — rules + intel
- Same check-url for web and extension
- HTTPS does not mean safe
- Urgency alone is not phishing
- Backend down is not SAFE
- No page passwords collected
- METRIC NOT YET AVAILABLE
- Biggest limit: no trained model + URL-centric analysis

Demo: Scan google.com, then paypa1.com or sbi-login.xyz, in both the website and the extension. Scores must match.

For the full 120 viva answers, open docs/JURY_VIVA_QUESTION_BANK.md